Sesión 14

Gestión de Errores, Recuperación y Arquitecturas Híbridas

Carlos César Sánchez Coronel

BASES DE DATOS

2026

Contenido general

1. Introducción
2. Operaciones de Alto Riesgo y Prevención
3. Recuperación ante Desastres (DRP)
4. Desafíos en la Nube y Data Drift
5. Arquitecturas Híbridas
6. Sincronización y Consistencia
7. Ejercicios Resueltos
8. Glosario

Contenido de la sección

1. Introducción
2. Operaciones de Alto Riesgo y Prevención
3. Recuperación ante Desastres (DRP)
4. Desafíos en la Nube y Data Drift
5. Arquitecturas Híbridas
6. Sincronización y Consistencia
7. Ejercicios Resueltos
8. Glosario

Objetivos de la sesión

- Conocer las operaciones de alto riesgo y las mejores prácticas para evitarlas.
- Entender los mecanismos de recuperación ante desastres: backups, snapshots, point-in-time recovery.
- Analizar los desafíos específicos de la nube: latencia, costos de egreso, data drift.
- Diseñar arquitecturas híbridas que integren motores SQL, NoSQL y vectoriales.
- Aprender a orquestar múltiples bases de datos con contenedores e infraestructura como código.
- Implementar sincronización entre sistemas mediante CDC y triggers.
- Resolver ejercicios prácticos que integran todos los conceptos.

Contenido de la sección

1. Introducción
2. Operaciones de Alto Riesgo y Prevención
3. Recuperación ante Desastres (DRP)
4. Desafíos en la Nube y Data Drift
5. Arquitecturas Híbridas
6. Sincronización y Consistencia
7. Ejercicios Resueltos
8. Glosario

DROP vs TRUNCATE vs DELETE

Operación	¿Elimina estructura?	¿Registra por fila?
DROP	Sí	No (metadata)
TRUNCATE	No	No (desasigna páginas)
DELETE sin WHERE	No	Sí (puede ser lento)

- DROP: irreversible sin backup.
- TRUNCATE: rápido, pero no se puede deshacer con ROLLBACK en muchos motores.
- DELETE: transaccional, pero genera muchos logs.

Buenas prácticas para prevenir desastres

- Siempre realizar un `SELECT` antes de un `DELETE` o `UPDATE` para verificar la condición.
- Usar transacciones: `BEGIN; DELETE ...; ROLLBACK;` para probar.
- En entornos críticos, revocar permisos de `DROP` y `TRUNCATE` para usuarios regulares.
- Implementar **soft delete** (columna `activo`) en lugar de eliminación física.
- Usar `sql_safe_updates` en MySQL (requiere WHERE con key).
- Auditoría de consultas con herramientas como `pgaudit`.

Ejemplo: Soft Delete

```sql
ALTER TABLE productos ADD COLUMN activo BOOLEAN DEFAULT TRUE;

-- En lugar de DELETE
UPDATE productos SET activo = FALSE WHERE id = 123;

-- Las consultas ignoran productos inactivos
SELECT * FROM productos WHERE activo = TRUE;
```

Contenido de la sección

1. Introducción
2. Operaciones de Alto Riesgo y Prevención
3. Recuperación ante Desastres (DRP)
4. Desafíos en la Nube y Data Drift
5. Arquitecturas Híbridas
6. Sincronización y Consistencia
7. Ejercicios Resueltos
8. Glosario

RTO y RPO

RTO Recovery Time Objective: tiempo máximo para restaurar el servicio.

RPO Recovery Point Objective: pérdida máxima de datos aceptable (expresada en tiempo).

- RPO bajo requiere replicación continua o backups incrementales frecuentes.
- RTO bajo requiere procedimientos automatizados (failover automático).

Backups y Snapshots

Backups lógicos Volcados de datos en SQL (ej. `pg_dump`). Portables, lentos.

Backups físicos Copias del sistema de archivos (snapshots EBS). Rápidos, dependientes del proveedor.

Backups incrementales Solo guardan cambios desde el último backup. Ahorran espacio.

Point-in-Time Recovery (PITR)

- Permite restaurar la base a un momento exacto antes de un incidente.
- Requiere:
  - Backup completo (basebackup).
  - Archivos WAL (Write-Ahead Logs) desde ese backup.
  - Configuración de archiving y recovery target.

Ejemplo: Configuración de PITR en PostgreSQL

```
# Configurar archiving en postgresql.conf
wal_level = replica
archive_mode = on
archive_command = 'cp %p /wal_archive/%f'
archive_timeout = 60

# Backup base
pg_basebackup -D /backup/base -F t -z -P

# En postgresql.auto.conf para recuperación:
restore_command = 'cp /wal_archive/%f %p'
recovery_target_time = '2025-03-20 14:30:00'
```

Al iniciar PostgreSQL, aplica WAL hasta ese momento.

Contenido de la sección

1. Introducción
2. Operaciones de Alto Riesgo y Prevención
3. Recuperación ante Desastres (DRP)
4. Desafíos en la Nube y Data Drift
5. Arquitecturas Híbridas
6. Sincronización y Consistencia
7. Ejercicios Resueltos
8. Glosario

Latencia y Costos de Egreso

- Mover datos entre regiones o fuera del proveedor tiene costo (egress).
- Estrategias:
  - Mantener datos en la misma región.
  - Comprimir antes de transferir.
  - Usar redes privadas (VPC peering).

Data Drift (Deriva de Datos)

- Cambio en la distribución de los datos de entrada que degrada modelos de IA.
- Se detecta comparando estadísticas (media, desviación) o tests como Kolmogorov-Smirnov.
- Herramientas: Evidently, Great Expectations, WhyLabs.

Detección de data drift con Python

```python
import numpy as np
from scipy.stats import ks_2samp

train = np.random.normal(100, 15, 1000)
prod = np.random.normal(105, 18, 500)

stat, p_value = ks_2samp(train, prod)
alpha = 0.05
if p_value < alpha:
    print("Drift detectado (p=%.4f)" % p_value)
else:
    print("No hay evidencia de drift")
```

Contenido de la sección

1. Introducción
2. Operaciones de Alto Riesgo y Prevención
3. Recuperación ante Desastres (DRP)
4. Desafíos en la Nube y Data Drift
5. Arquitecturas Híbridas
6. Sincronización y Consistencia
7. Ejercicios Resueltos
8. Glosario

Capas en una arquitectura políglota

Transaccional (SQL) Datos maestros, ACID. Ej: PostgreSQL.

Velocidad (NoSQL) Caché, sesiones, baja latencia. Ej: Redis.

Semántica (Vectorial) Búsqueda por similitud, embeddings. Ej: pgvector, Milvus.

Orquestación con Docker Compose

```yaml
version: '3'
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: admin
      POSTGRES_DB: test
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net
  redis:
    image: redis:7
    networks:
      - app-net
  app:
    build: ./app
    depends_on:
      - postgres
      - redis
    networks:
      - app-net
networks:
  app-net:
    driver: bridge
volumes:
  pgdata:
```

Infraestructura como Código (IaC)

- Definir infraestructura de forma declarativa (Terraform, CloudFormation).
- Ventajas: replicable, versionable, evita errores manuales.

Ejemplo Terraform para RDS

Contenido de la sección

1. Introducción
2. Operaciones de Alto Riesgo y Prevención
3. Recuperación ante Desastres (DRP)
4. Desafíos en la Nube y Data Drift
5. Arquitecturas Híbridas
6. Sincronización y Consistencia
7. Ejercicios Resueltos
8. Glosario

Sincronización con Triggers en SQL

```sql
CREATE TABLE cambios_productos (
    id SERIAL PRIMARY KEY,
    producto_id INT,
    nombre TEXT,
    operacion CHAR(1),
    procesado BOOLEAN DEFAULT FALSE
);

CREATE OR REPLACE FUNCTION log_producto() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO cambios_productos (producto_id, nombre, operacion)
    VALUES (NEW.id, NEW.nombre, TG_OP);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_productos
AFTER INSERT OR UPDATE OR DELETE ON productos
FOR EACH ROW EXECUTE FUNCTION log_producto();
```

Un worker externo lee la tabla y actualiza Redis/vectorial.

Worker Python para sincronizar con Redis

```
import psycopg2, redis, time

conn_pg = psycopg2.connect(dbname="test", user="postgres")
cur_pg = conn_pg.cursor()
r = redis.Redis(host='localhost', port=6379, decode_responses=True)

while True:
    cur_pg.execute("SELECT id, producto_id, nombre, operacion FROM cambios_productos WHERE
        procesado = FALSE LIMIT 1")
    row = cur_pg.fetchone()
    if row:
        cambio_id, prod_id, nombre, op = row
        if op in ('INSERT', 'UPDATE'):
            r.set(f"producto:{prod_id}", nombre)
        elif op == 'DELETE':
            r.delete(f"producto:{prod_id}")
        cur_pg.execute("UPDATE cambios_productos SET procesado = TRUE WHERE id = %s", (cambio_id
            ,))
        conn_pg.commit()
        print(f"Sincronizado {prod_id}")
    time.sleep(1)
```

Change Data Capture (CDC)

- Debezium captura cambios del log de transacciones.
- Publica en Kafka; consumidores actualizan bases vectoriales.

Consistencia Eventual

- La replicación asíncrona introduce latencia.
- Aplicaciones deben tolerar que la búsqueda no incluya los últimos cambios.
- Estrategias: marcar datos como "no indexados"hasta sincronizar, usar TTL.

Contenido de la sección

1. Introducción
2. Operaciones de Alto Riesgo y Prevención
3. Recuperación ante Desastres (DRP)
4. Desafíos en la Nube y Data Drift
5. Arquitecturas Híbridas
6. Sincronización y Consistencia
7. Ejercicios Resueltos
8. Glosario

Ejercicio 1: Prevención de DROP accidental

Enunciado: En PostgreSQL, crear un trigger que impida ejecutar DROP TABLE en una tabla crítica.

Solución con Event Trigger

```
CREATE OR REPLACE FUNCTION abort_drop()
RETURNS event_trigger AS $$
BEGIN
    RAISE EXCEPTION 'No se permite DROP en esta base de datos';
END;
$$ LANGUAGE plpgsql;

CREATE EVENT TRIGGER abort_drop_trigger
ON sql_drop
EXECUTE FUNCTION abort_drop();
```

(Nota: impide cualquier DROP; también se puede revocar permisos específicos)

Ejercicio 2: Configurar PITR en PostgreSQL con Docker

Enunciado: Levantar PostgreSQL con archiving WAL y restaurar a un momento antes de un error.

Pasos resumidos

```
# Crear contenedor con archiving
docker run -d --name pg-pitr -e POSTGRES_PASSWORD=admin -v pgdata:/var/lib/postgresql/data -v
    walarchive:/wal_archive postgres:15 -c wal_level=replica -c archive_mode=on -c
    archive_command='cp %p /wal_archive/%f' -c archive_timeout=60

# Simular datos y luego DROP
docker exec -it pg-pitr psql -U postgres -c "CREATE TABLE test (id int); INSERT INTO test VALUES
    (1); SELECT pg_switch_wal();"

# Restaurar: detener contenedor, preparar recovery, iniciar nuevo contenedor...
```

Ejercicio 3: Integración PostgreSQL + Redis con CDC simulado

Enunciado: Usar trigger y worker Python para sincronizar productos con Redis.

Solución (ver diapositivas anteriores)

(El código ya se mostró en la sección de sincronización).

Ejercicio 4: Despliegue híbrido con Docker Compose

Enunciado: Crear docker-compose.yml con PostgreSQL, Redis y una app Python.

Solución (ver diapositiva de orquestación)

(El YAML ya se mostró).

Ejercicio 5: Detección de data drift con Python

Enunciado: Usar test de Kolmogorov-Smirnov para detectar drift.

Solución (ver diapositiva de data drift)

(El código Python ya se mostró).

Contenido de la sección

1. Introducción
2. Operaciones de Alto Riesgo y Prevención
3. Recuperación ante Desastres (DRP)
4. Desafíos en la Nube y Data Drift
5. Arquitecturas Híbridas
6. Sincronización y Consistencia
7. Ejercicios Resueltos
8. Glosario

Términos Clave

DRP	Disaster Recovery Plan, plan de recuperación ante desastres.
PITR	Point-in-Time Recovery, recuperación a un momento exacto.
WAL	Write-Ahead Log, registro de transacciones en PostgreSQL.
CDC	Change Data Capture, captura de cambios en bases de datos.
IaC	Infrastructure as Code, gestión de infraestructura mediante código.
Snapshot	Imagen instantánea de un sistema de archivos.
Data Drift	Cambio en la distribución de datos que afecta modelos de IA.
Egreso	Transferencia de datos fuera de un proveedor cloud (costo).
Consistencia eventual	Los datos se vuelven consistentes después de un tiempo.
Políglota	Uso de múltiples tipos de bases de datos en una misma aplicación.

¿Preguntas?

¡Gracias por su atención!